

1. Write a C program to find the growth in national consumption for five years using Distributed Lag Model given below:

$$I = 2 + 0.1 Y_{-1}$$

$$Y = 45.45 + 2.27 (I + G)$$

$$T = 0.2 Y$$

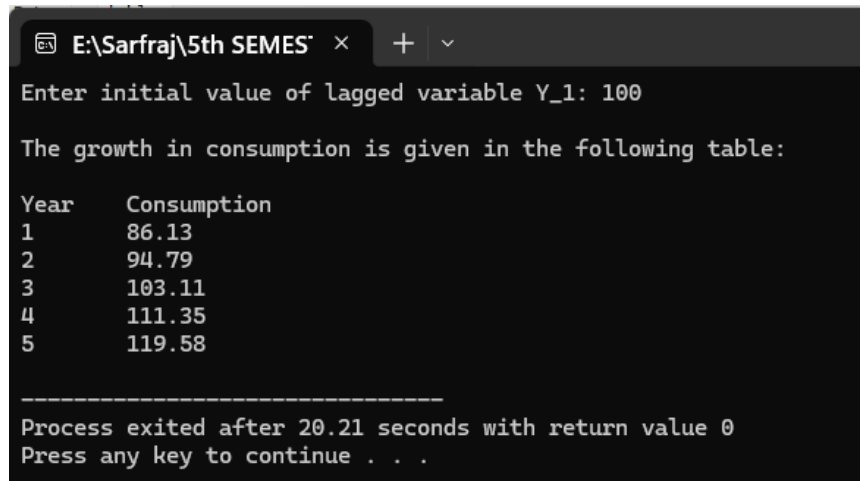
$$C = 20 + 0.7 (Y - T)$$

Assume the initial value of Y_{-1} is 80 and take the governmental expenditure in the 5 years to be as follows:

Year	G
1	20
2	25
3	30
4	35
5	40

Source code:

```
#include <stdio.h>
int main()
{
    float Y_1, Y, I, T, C[5];
    float G[5] = {20, 25, 30, 35, 40};
    int i;
    printf("Enter initial value of lagged variable Y_1: ");
    scanf("%f", &Y_1);
    printf("\nThe growth in consumption is given in the following table:\n");
    printf("\nYear\tConsumption\n");
    for(i = 0; i < 5; i++)
    {
        I = 2 + 0.1 * Y_1;
        Y = 45.45 + 2.27 * (I + G[i]);
        T = 0.2 * Y;
        C[i] = 20 + 0.7 * (Y - T);
        printf("%d\t%.2f\n", i + 1, C[i]);
        Y_1 = Y;
    }
    return 0;
```



The screenshot shows a Windows command prompt window titled "E:\Sarfraj\5th SEMES". The program prompts the user to enter the initial value of the lagged variable Y_1, which is 100. It then displays a table of consumption values for years 1 through 5. The values are 86.13, 94.79, 103.11, 111.35, and 119.58 respectively. The program exits after 20.21 seconds with a return value of 0.

```
E:\Sarfraj\5th SEMES>
Enter initial value of lagged variable Y_1: 100

The growth in consumption is given in the following table:

Year    Consumption
1        86.13
2        94.79
3        103.11
4        111.35
5        119.58

-----
Process exited after 20.21 seconds with return value 0
Press any key to continue . . .
```

2. Customers arrive in a bank according to a Poisson's process with mean inter arrival time of 10 minutes. Customers spend an average of 5 minutes on the single available counter, and leave.

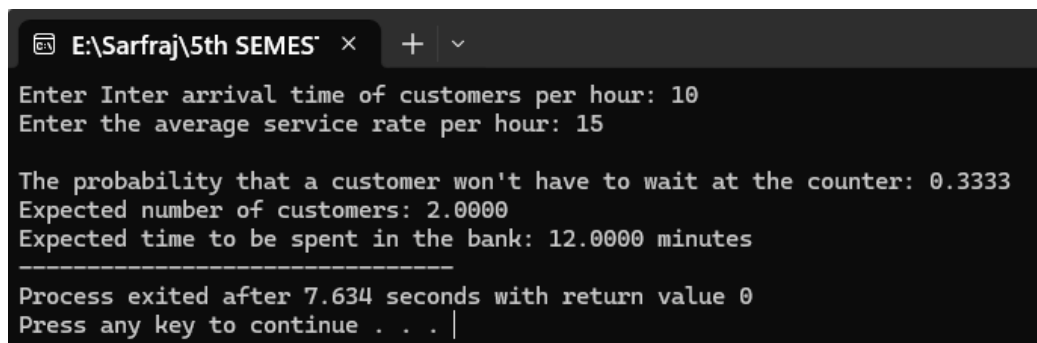
Write a program in C to find:

- I. Probability that a customer will not have to wait at the counter.**
- II. Expected number of customers in the bank.**
- III. Time can a customer expect to spend in the bank**

Source code:

```
#include <stdio.h>

int main()
{
    float l, m;
    float p, e, et;
    printf("Enter Inter arrival time of customers per hour: ");
    scanf("%f", &l);
    printf("Enter the average service rate per hour: ");
    scanf("%f", &m);
    if (m <= l)
    {
        printf("\nSystem is unstable because service rate must be greater than arrival rate.\n");
        return 0;
    }
    p = 1 - (l / m);
    e = l / (m - l);
    et = 1 / (m - l);
    et = et * 60; // Convert hours to minutes
    printf("\nThe probability that a customer won't have to wait at the counter: %.4f", p);
    printf("\nExpected number of customers: %.4f", e);
    printf("\nExpected time to be spent in the bank: %.4f minutes", et);
    return 0;
}
```



```
E:\Sarfraj\5th SEMES x + v
Enter Inter arrival time of customers per hour: 10
Enter the average service rate per hour: 15

The probability that a customer won't have to wait at the counter: 0.3333
Expected number of customers: 2.0000
Expected time to be spent in the bank: 12.0000 minutes
-----
Process exited after 7.634 seconds with return value 0
Press any key to continue . . . |
```

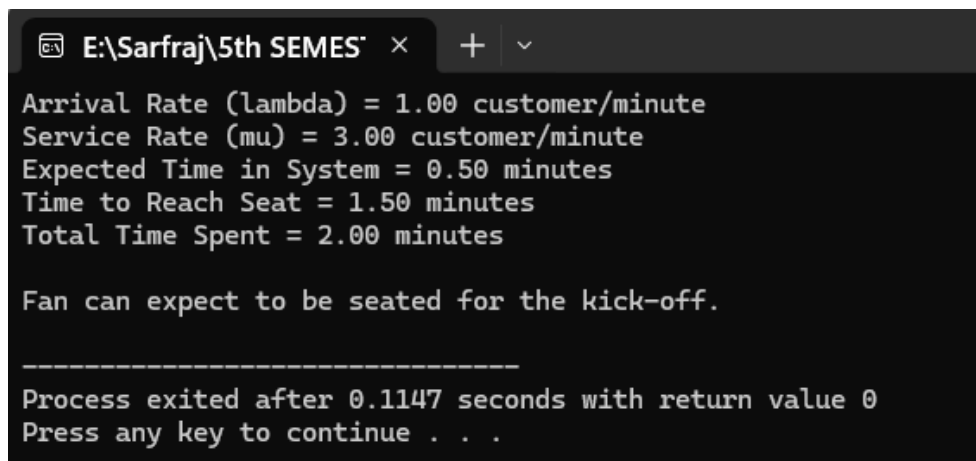
3. At the ticket counter of football stadium, people come in queue and purchase tickets. Arrival rate of customers is 1/min. It takes at the average 20 seconds to purchase the ticket.

WAP in C to calculate total time spent by a sports fan to be seated in his seat, if it takes 1.5 minutes to reach the correct seat after purchasing the ticket. if a fan comes exactly before 2 minutes before the game starts, can sports fan expects to be seated for the kick-off?

Source code:

```
#include <stdio.h>

int main()
{
    float lambda, serviceTime, mu;
    float expectedTime, reachSeat, totalTime;
    lambda = 1.0;    // customers per minute
    serviceTime = 20.0; // seconds
    mu = 60.0 / serviceTime; // customers per minute
    expectedTime = 1.0 / (mu - lambda);
    reachSeat = 1.5;    // minutes
    totalTime = expectedTime + reachSeat;
    printf("Arrival Rate (lambda) = %.2f customer/minute\n", lambda);
    printf("Service Rate (mu) = %.2f customer/minute\n", mu);
    printf("Expected Time in System = %.2f minutes\n", expectedTime);
    printf("Time to Reach Seat = %.2f minutes\n", reachSeat);
    printf("Total Time Spent = %.2f minutes\n", totalTime);
    if(totalTime <= 2.0)
        printf("\nFan can expect to be seated for the kick-off.\n");
    else
        printf("\nFan may miss the kick-off.\n");
    return 0;
}
```



```
E:\Sarfraj\5th SEMES x + v
Arrival Rate (lambda) = 1.00 customer/minute
Service Rate (mu) = 3.00 customer/minute
Expected Time in System = 0.50 minutes
Time to Reach Seat = 1.50 minutes
Total Time Spent = 2.00 minutes

Fan can expect to be seated for the kick-off.

-----
Process exited after 0.1147 seconds with return value 0
Press any key to continue . . .
```

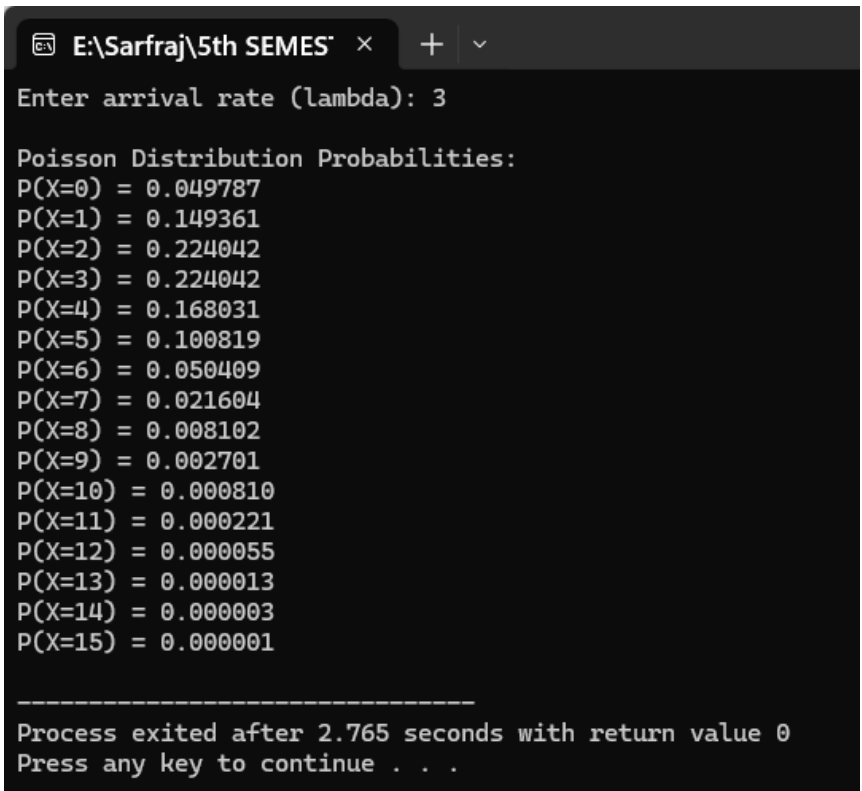
4. In a single pump service station, vehicles arrive for fuelling with an average of 5 minutes between arrivals. If an hour is taken as unit of time, cars arrive according to Poisson's process with an average of $\lambda = 12$ cars/hr.

Write a C program to generate Poisson distribution for $x = 0, 1, 2, \dots, 15$

$$f(x) = \Pr(X=x) = \frac{e^{-\lambda} \lambda^x}{x!} = \frac{e^{-12} 12^x}{x!}, \begin{cases} x = 0, 1, 2, \dots \\ \lambda > 0 \end{cases}$$

Source code:

```
#include <stdio.h>
#include <math.h>
int main()
{
    float l, pr, a;
    int x, i;
    printf("Enter arrival rate (lambda): ");
    scanf("%f", &l);
    printf("\nPoisson Distribution Probabilities:\n");
    for (x = 0; x < 16; x++)
    {
        a = 1;
        for (i = 1; i <= x; i++)
        {
            a *= i;
        }
        pr = (exp(-l) * pow(l, x)) / a;
        printf("P(X=%d) = %.6f\n", x, pr);
    }
    return 0;
}
```



The screenshot shows a Windows command prompt window titled "E:\Sarfraj\5th SEMES". The user has entered the arrival rate (lambda) as 3. The program outputs the Poisson distribution probabilities for x from 0 to 15. The probabilities are displayed in scientific notation, rounded to six decimal places. At the bottom, the program indicates it exited after 2.765 seconds with a return value of 0 and prompts the user to press any key to continue.

```
E:\Sarfraj\5th SEMES x + v
Enter arrival rate (lambda): 3

Poisson Distribution Probabilities:
P(X=0) = 0.049787
P(X=1) = 0.149361
P(X=2) = 0.224042
P(X=3) = 0.224042
P(X=4) = 0.168031
P(X=5) = 0.100819
P(X=6) = 0.050409
P(X=7) = 0.021604
P(X=8) = 0.008102
P(X=9) = 0.002701
P(X=10) = 0.000810
P(X=11) = 0.000221
P(X=12) = 0.000055
P(X=13) = 0.000013
P(X=14) = 0.000003
P(X=15) = 0.000001

-----
Process exited after 2.765 seconds with return value 0
Press any key to continue . . .
```

5.The probabilities of weather conditions (modeled as either rainy or sunny), given the weather on the preceding day, can be represented by a transition matrix:

0.9 0.1

0.5 0.5

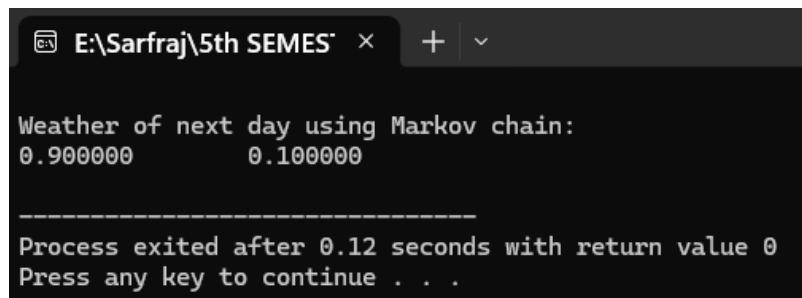
The weather on day 0 is known to be sunny. This is represented by a vector in which the "sunny" entry is 100%, and the "rainy" entry is 0%: (1 0) .

Write a C program to find the weather of the next day by using Markov Chain Method.

Source code:

```
#include <stdio.h>

int main() {
    float transMat[2][2] = {{0.9, 0.1}, {0.5, 0.5}};
    float vect[1][2] = {{1.0, 0.0}};
    float result[1][2] = {0.0, 0.0};
    int i, j, k;
    for (i = 0; i < 1; i++) {
        for (j = 0; j < 2; j++) {
            for (k = 0; k < 2; k++) {
                result[i][j] += vect[i][k] * transMat[k][j];
            }
        }
    }
    printf("\nWeather of next day using Markov chain:\n");
    for (i = 0; i < 1; i++) {
        for (j = 0; j < 2; j++) {
            printf("%f\t", result[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```



```
E:\Sarfraj\5th SEMES x + v

Weather of next day using Markov chain:
0.900000      0.100000

-----
Process exited after 0.12 seconds with return value 0
Press any key to continue . . .
```

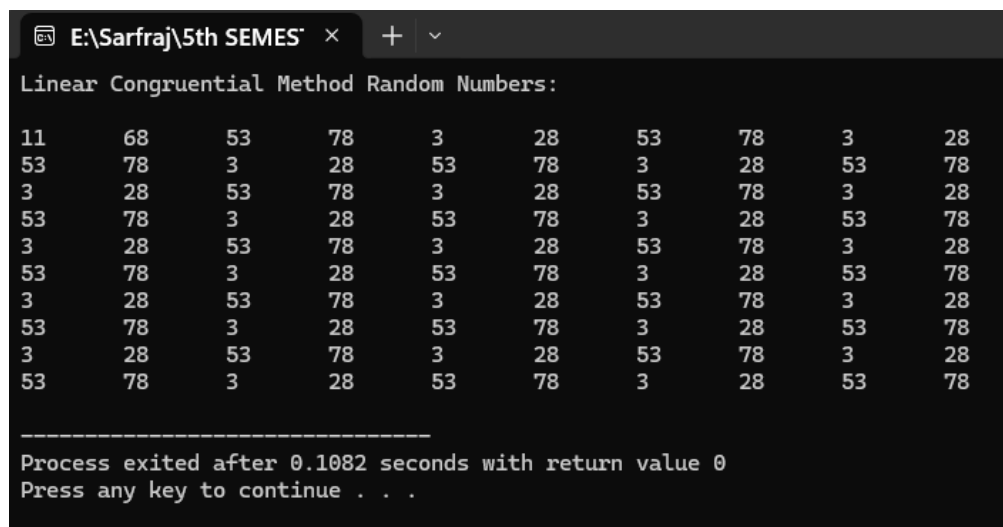
6.What is random number and what are its properties. WAP in C to generate 100 random numbers using Linear Congruential Method where $X_0=11$, $m=100$, $a = 5$ and $c = 13$.

Source code:

```
#include <stdio.h>

#define SIZE 100

int main()
{
    int x[SIZE];
    int m = 100, a = 5, c = 13;
    int i;
    x[0] = 11;
    for(i = 0; i < SIZE - 1; i++)
    {
        x[i + 1] = (a * x[i] + c) % m;
    }
    printf("Linear Congruential Method Random Numbers:\n\n");
    for(i = 0; i < SIZE; i++)
    {
        printf("%d\t", x[i]);
        // print new line after every 10 numbers
        if((i + 1) % 10 == 0)
        {
            printf("\n");
        }
    }
    return 0;
}
```



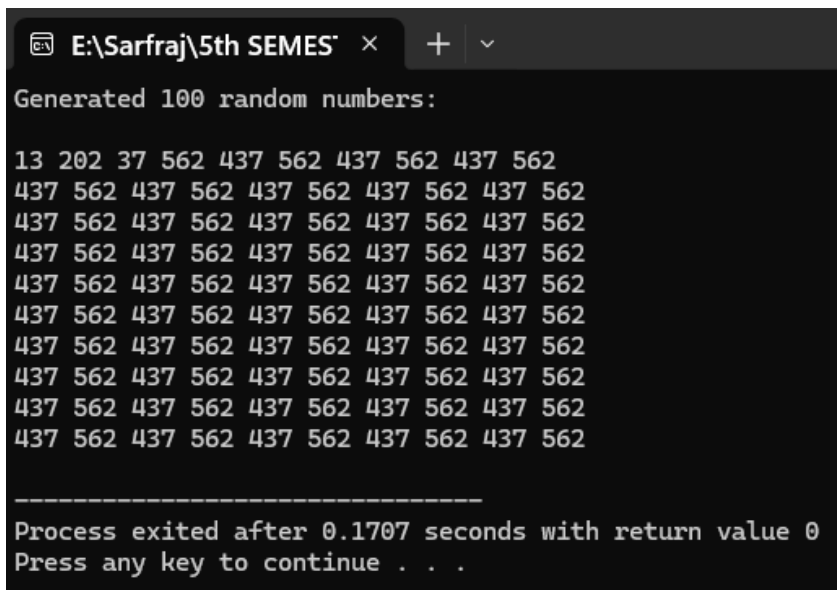
```
E:\Sarfraj\5th SEMES' x + v
Linear Congruential Method Random Numbers:
11    68    53    78    3    28    53    78    3    28
53    78    3    28    53    78    3    28    53    78
3     28    53    78    3    28    53    78    3    28
53    78    3    28    53    78    3    28    53    78
3     28    53    78    3    28    53    78    3    28
53    78    3    28    53    78    3    28    53    78
3     28    53    78    3    28    53    78    3    28
53    78    3    28    53    78    3    28    53    78
3     28    53    78    3    28    53    78    3    28
53    78    3    28    53    78    3    28    53    78

-----
Process exited after 0.1082 seconds with return value 0
Press any key to continue . . .
```

7.WAP in C to generate 100 random numbers using Multiplicative Congruential Method where $X_0=13$, $m=1000$, $a=15$ and $c=7$.

Source code:

```
#include <stdio.h>
#define SIZE 100
int main() {
    long x[SIZE];
    long m = 1000 , a = 15, c = 7;
    int i;
    x[0] = 13;
    for (i = 0; i < SIZE - 1; i++) {
        x[i + 1] = (a * x[i] + c) % m;
    }
    printf("Generated 100 random numbers:\n\n");
    for (i = 0; i < SIZE; i++) {
        printf("%ld ", x[i]);
        // print new line after every 10 numbers
        if((i + 1) % 10 == 0)
        {
            printf("\n");
        }
    }
    return 0;
}
```



```
E:\Sarfraj\5th SEMES x + v
Generated 100 random numbers:

13 202 37 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562
437 562 437 562 437 562 437 562 437 562

-----
Process exited after 0.1707 seconds with return value 0
Press any key to continue . . .
```

GPSS

8. Create a GPSS model and program to simulate a barber shop for a day (9 a.m. to 4 p.m.), where a customer enters the shop every 10 ± 2 minutes and a barber takes 13 ± 2 minutes for a haircut.

GENERATE 10,2

QUEUE SEAT

SEIZE BARBER

DEPART SEAT

ADVANCE 13,2

RELEASE BARBER

TERMINATE 1

GENERATE 420

TERMINATE 1

START 1

GENERATE

QUEUE

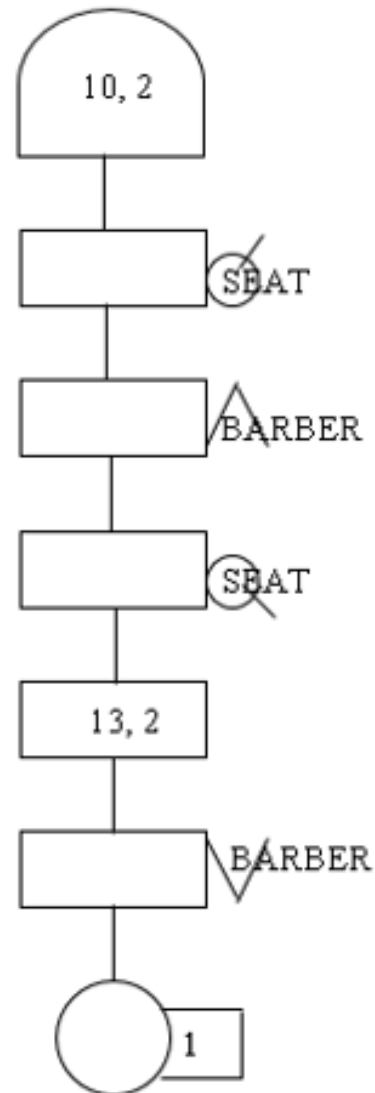
SEIZE

DEPART

ADVANCE

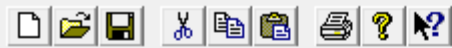
RELEASE

TERMINATE



GPSS World - [Untitled Model 1.1.1 - REPORT]

File Edit Search View Command Window Help



GPSS World Simulation Report - Untitled Model 1.1.1

Friday, June 19, 2026 21:57:58

START TIME	END TIME	BLOCKS	FACILITIES	STORAGES
0.000	25.893	9	1	0

NAME	VALUE
BARBER	10001.000
SEAT	10000.000

LABEL	LOC	BLOCK TYPE	ENTRY	COUNT	CURRENT	COUNT	RETRY
	1	GENERATE	2		0	0	
	2	QUEUE	2		0	0	
	3	SEIZE	2		1	0	
	4	DEPART	1		0	0	
	5	ADVANCE	1		0	0	
	6	RELEASE	1		0	0	
	7	TERMINATE	1		0	0	
	8	GENERATE	0		0	0	
	9	TERMINATE	0		0	0	

FACILITY	ENTRIES	UTIL.	AVE. TIME	AVAIL.	OWNER	PEND	INTER	RETRY	DELAY
BARBER	2	0.561	7.262	1	3	0	0	0	0

QUEUE	MAX	CONT.	ENTRY	ENTRY(0)	AVE.CONT.	AVE.TIME	AVE.(-0)	RETRY
SEAT	1	1	2	1	0.132	1.707	3.413	0

CEC	XN	PRI	M1	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
	3	0	22.480	3	3	4		

FEC	XN	PRI	BDT	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
	4	0	31.522	4	0	1		
	2	0	420.000	2	0	8		

For Help, press F1

Report is Complete.

9. A machine tool in a manufacturing shop is turning out parts at the rate of every 5 minutes. When they are finished, the parts are sent to an inspector, who takes 4 ± 3 minutes to examine each one and rejects 15% of the parts. Draw and explain a block diagram for it and write a GPSS program to simulate using the concept of FACILITY.

GENERATE 5,0

QUEUE INSQ

SEIZE INSPECTOR

DEPART INSQ

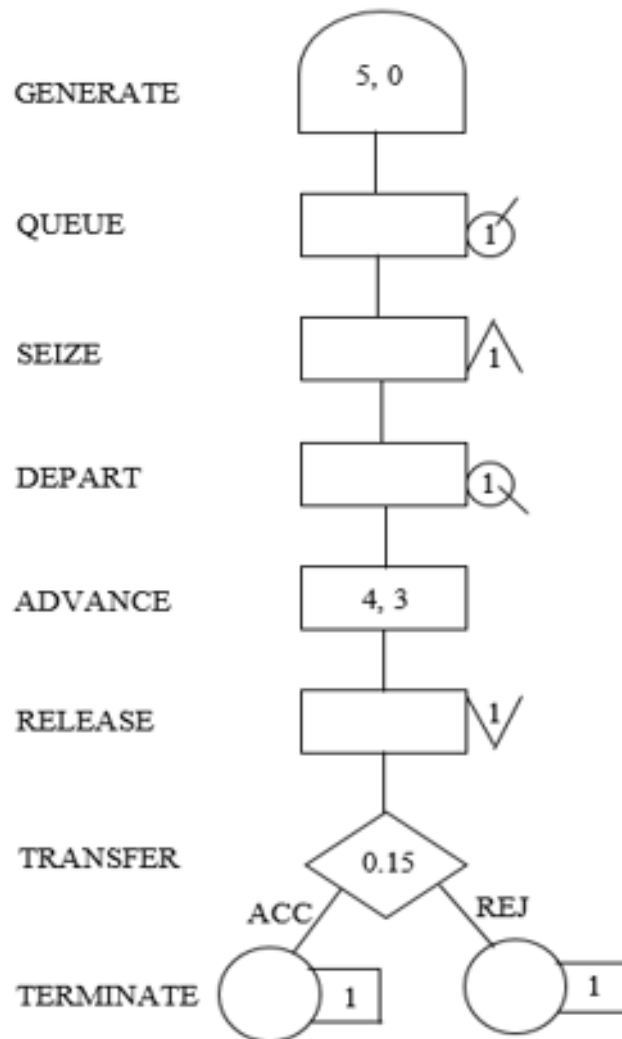
ADVANCE 4,3

RELEASE INSPECTOR

TRANSFER 0.15,REJ,ACC

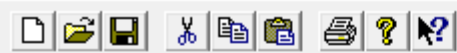
ACC TERMINATE 1

REJ TERMINATE 1



GPSS World - [Untitled Model 1.1.1 - REPORT]

File Edit Search View Command Window Help



GPSS World Simulation Report - Untitled Model 1.1.1

Friday, June 19, 2026 22:20:18

START TIME	END TIME	BLOCKS	FACILITIES	STORAGES
0.000	15.326	9	1	0

NAME	VALUE
ACC	8.000
INSPECTOR	10001.000
INSQ	10000.000
REJ	9.000

LABEL	LOC	BLOCK TYPE	ENTRY	COUNT	CURRENT	COUNT	RETRY
	1	GENERATE		3		0	0
	2	QUEUE		3		0	0
	3	SEIZE		3		1	0
	4	DEPART		2		0	0
	5	ADVANCE		2		0	0
	6	RELEASE		2		0	0
	7	TRANSFER		2		0	0
ACC	8	TERMINATE		1		0	0
REJ	9	TERMINATE		1		0	0

FACILITY	ENTRIES	UTIL.	AVE. TIME	AVAIL.	OWNER	PEND	INTER	RETRY	DELAY
INSPECTOR	3	0.674	3.442	1	3	0	0	0	0

QUEUE	MAX	CONT.	ENTRY	ENTRY(0)	AVE.CONT.	AVE.TIME	AVE.(-0)	RETRY
INSQ	1	1	3	1	0.105	0.537	0.806	0

CEC	XN	PRI	M1	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
	3	0	15.000	3	3	4		

FEC	XN	PRI	BDT	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
	4	0	20.000	4	0	1		

For Help, press F1

Report is Complete.

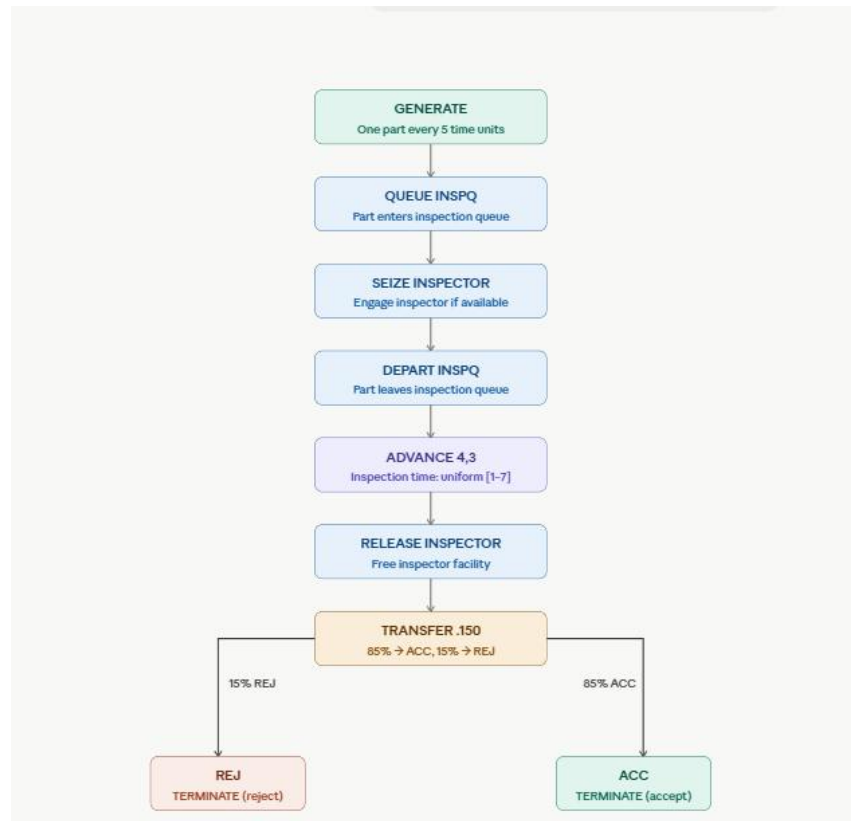
10.A GENERATE block is used to represent the output of the machine by creating one transaction every five units of time. A QUEUE block places the transaction in the queue and SEIZE block allows a transaction to engage a facility if it is available. If more than one inspector is available, the transaction leaves the queue which is denoted by DEPART block and enters into ADVANCE block. An ADVANCE block with a mean of 4 and modifier of 3 is used to represent inspection. The time spent on inspection will therefore be any one of the values 1, 2, 3, 4, 5, 6 or 7, with equal probability given to each value. Upon completion of the inspection, RELEASE block allows a transaction to disengage the facility and transaction go to a TRANSFER block with a selection factor of 0.15, so that 85% of the parts go to the next location called ACC, to represent accepted parts and 15% go to another location called REJ to represent rejects. Both locations reached from the TRANSFER block are TERMINATE blocks.

GENERATE 5,0,,,100
QUEUE INSPQ
SEIZE INSPECTOR
DEPART INSPQ
ADVANCE 4,3
RELEASE INSPECTOR
TRANSFER 0.15,REJ,ACC

ACC TERMINATE 1

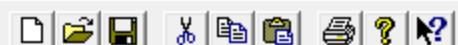
REJ TERMINATE 1

START 100



GPSS World - [Untitled Model 2.1.1 - REPORT]

File Edit Search View Command Window Help



GPSS World Simulation Report - Untitled Model 2.1.1

Friday, June 19, 2026 22:46:14

START TIME	END TIME	BLOCKS	FACILITIES	STORAGES
0.000	505.643	9	1	0

NAME	VALUE
ACC	8.000
INSPECTOR	10001.000
INSQ	10000.000
REJ	9.000

LABEL	LOC	BLOCK TYPE	ENTRY COUNT	CURRENT COUNT	RETRY
	1	GENERATE	101	0	0
	2	QUEUE	101	0	0
	3	SEIZE	101	1	0
	4	DEPART	100	0	0
	5	ADVANCE	100	0	0
	6	RELEASE	100	0	0
	7	TRANSFER	100	0	0
ACC	8	TERMINATE	17	0	0
REJ	9	TERMINATE	83	0	0

FACILITY	ENTRIES	UTIL.	AVE. TIME	AVAIL.	OWNER	PEND	INTER	RETRY	DELAY
INSPECTOR	101	0.826	4.133	1	101	0	0	0	0

QUEUE	MAX CONT.	ENTRY	ENTRY(0)	AVE. CONT.	AVE. TIME	AVE. (-0)	RETRY
INSQ	2	1	101	42	0.206	1.029	1.762 0

CEC XN	PRI	M1	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
101	100	505.000	101	3	4		

FEC XN	PRI	BDT	ASSEM	CURRENT	NEXT	PARAMETER	VALUE
102	100	510.000	102	0	1		

For Help, press F1

Report is Complete.

11. Parts are being made at the rate of one every 10 minutes. They are of two types; A and B. Mixed randomly with 10% being type B. Separate inspectors inspect each type. Inspection time for A is 6 ± 2 minutes and for B is 10 ± 2 minutes. Both inspectors reject 10% of parts. Simulate for 100 parts.

```

GENERATE 10,,,100
TRANSFER .10,TYPEB,TYPEA

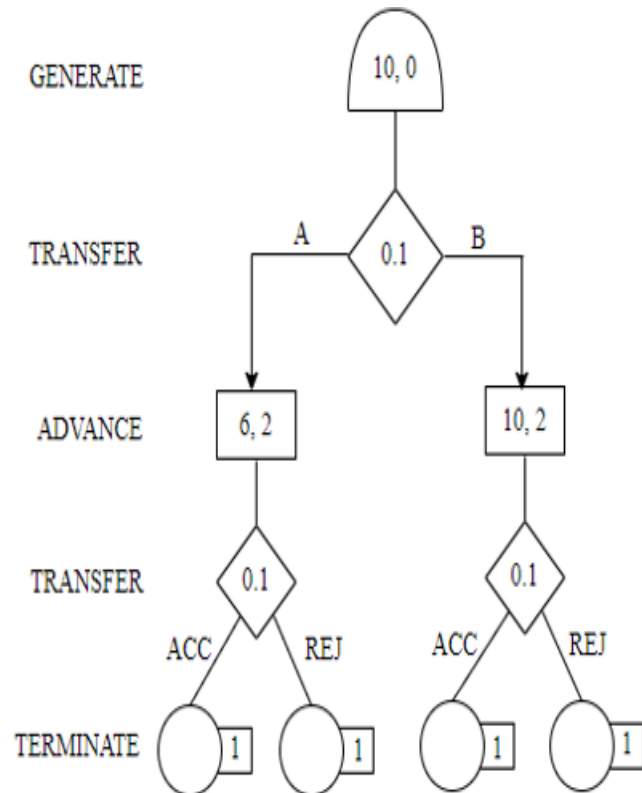
TYPEA QUEUE AQUEUE
  SEIZE INSPA
  DEPART AQUEUE
  ADVANCE 6,2
  RELEASE INSPA
  TRANSFER .10,REJA,ACCA

ACCA TERMINATE 1
REJA TERMINATE 1

TYPEB QUEUE BQUEUE
  SEIZE INSPB
  DEPART BQUEUE
  ADVANCE 10,2
  RELEASE INSPB
  TRANSFER .10,REJB,ACCB

ACCB TERMINATE 1
REJB TERMINATE 1
START 100

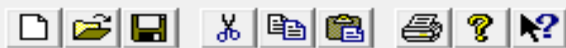
```





GPSS World - [Lab11.1.1 - REPORT]

File Edit Search View Command Window Help



GPSS World Simulation Report - Lab11.1.1

Friday, June 19, 2026 22:54:57

START TIME	END TIME	BLOCKS	FACILITIES	STORAGES
0.000	1008.573	18	2	0

NAME	VALUE
ACCA	9.000
ACCB	17.000
AQUEUE	10002.000
BQUEUE	10000.000
INSPA	10003.000
INSPB	10001.000
REJA	10.000
REJB	18.000
TYPEA	3.000
TYPEB	11.000

LABEL	LOC	BLOCK TYPE	ENTRY COUNT	CURRENT COUNT	RETRY
	1	GENERATE	100	0	0
	2	TRANSFER	100	0	0
TYPEA	3	QUEUE	11	0	0
	4	SEIZE	11	0	0
	5	DEPART	11	0	0
	6	ADVANCE	11	0	0
	7	RELEASE	11	0	0
	8	TRANSFER	11	0	0
ACCA	9	TERMINATE	2	0	0
REJA	10	TERMINATE	9	0	0
TYPEB	11	QUEUE	89	0	0
	12	SEIZE	89	0	0
	13	DEPART	89	0	0
	14	ADVANCE	89	0	0
	15	RELEASE	89	0	0
	16	TRANSFER	89	0	0
ACCB	17	TERMINATE	11	0	0
REJB	18	TERMINATE	78	0	0

FACILITY	ENTRIES	UTIL.	AVE. TIME	AVAIL.	OWNER	PEND	INTER	RETRY	DELAY
INSPB	89	0.871	9.871	1	0	0	0	0	0
INSPA	11	0.075	6.849	1	0	0	0	0	0

QUEUE	MAX CONT.	ENTRY	ENTRY(0)	AVE.CONT.	AVE.TIME	AVE.(-0)	RETRY	
BQUEUE	1	0	89	37	0.115	1.300	2.225	0
AQUEUE	1	0	11	11	0.000	0.000	0.000	0

For Help, press F1

Report is Complete.